

Coding agents in computational science

What AI can and can't do for our work, and a reproducible workflow built on it.

Eoin Murray · lab group talk

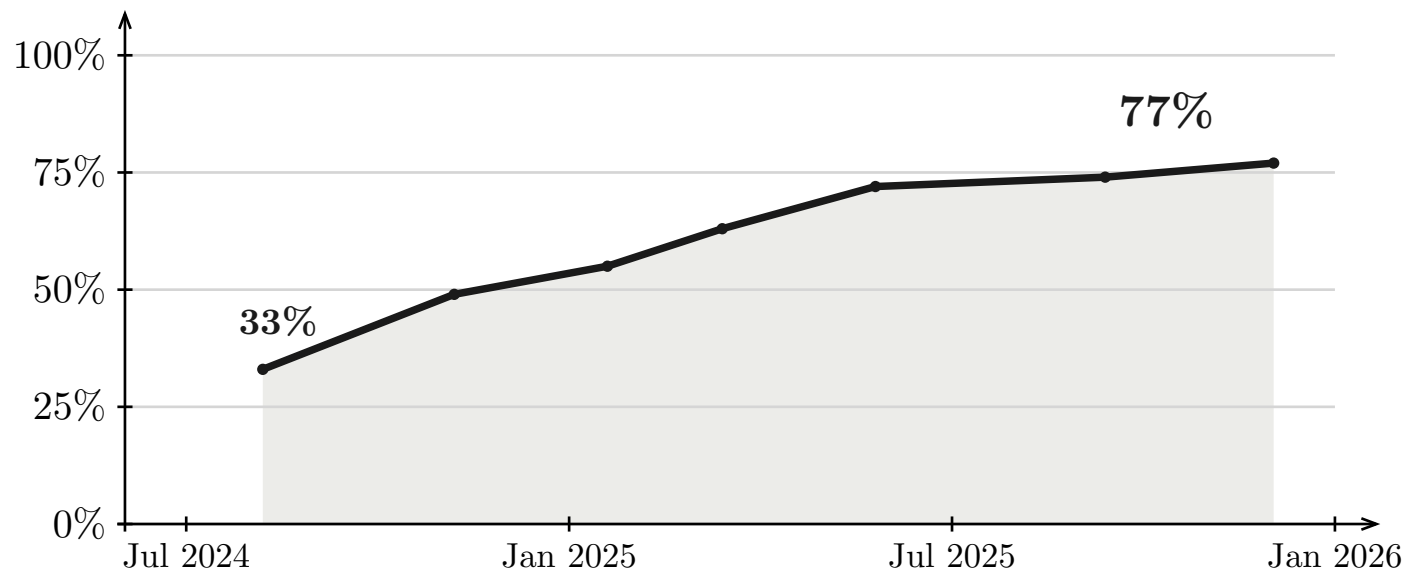
PART ONE

Coding agents

An observation

- **Software engineers** use coding agents like crazy.
- **Scientists**, far less so.

Coding agents are now useful



Share of real, human-validated GitHub issues an agent resolves end-to-end — SWE-bench Verified (swebench.com). Software bugs, not science.

autocomplete (2021) → chat (2023) → **agents that do real work (2025)**

What a coding agent is

- **Does the work**, not just autocomplete: reads, runs, edits, verifies.
- Driven in **plain language**, from your terminal.
- Many exist, commercial and open — no vendor lock-in.
- Only recently good enough — still uneven, still supervised.

More is possible

- Not the same work done faster — a **lower bar for what's worth doing at all**.
- A task that cost an afternoon, you skipped. At ten minutes, you just do it.
- So the skipped work happens: the extra **baseline**, the **ablation**, reproducing a **cited result**, the **README** nobody writes.
- You take on **more** — and things you'd never have bothered with.

Weaknesses of coding agents

- **Confidently wrong:** they fabricate APIs, numbers, and citations.
- **Comprehension debt:** they implement things you don't understand — and you ship them.
- **Weak judgement:** they won't tell you the experiment itself is a bad idea.
- **Drift & noise:** code and prose fall out of sync; they over-produce.
- **Brainrot:** prompt-and-wait is a slot machine — personally, I can't agent-code and read deeply the same day.

PART TWO

Demolab

Rails for the agent

Principles of demolab

1. `tools/` compute, `experiments/` do analysis and plots.
2. Each experiment is a runner (`.py`) and a write-up (`.typ`).
3. Tools and experiments talk through **data files**, never imports.
4. **Raw data is disposable** (`temp/`); the distilled record is committed (`artifacts/`).
5. Bring any stack.
6. **Agent-operated**: plain language in, you stay in the loop.
7. Provenance built in.

Typst: a modern LaTeX

LaTeX

- Slow, multi-pass compiles.
- Cryptic error messages.
- PDF only.
- Macro soup and package hell.

Typst

- **Instant**, incremental compiles.
- **Readable** errors with line numbers.
- One source → **web and PDF**.
- **Imports files**, no hardcoded numbers.

The same beautiful math: selectable MathML on the web, typeset in the PDF.

The shape of a demolab repo

<code>tools/</code>	the science – models & solvers
<code>experiments/</code>	the runners – <code>expNNN.py</code> per experiment
<code>writings/</code>	the writeups – one <code>.typ</code> per entry
<code>temp/</code>	the raw run data – gitignored, big
<code>artifacts/</code>	the distilled record – <code>data/</code> + <code>pdfs/</code> (committed)
<code>demolab.yaml</code>	branding + collections (optional)
<code>demolab-engine/</code>	the engine (black box you never edit)

Guides and runbooks

```
my-coding-agent ► HELP ■
```

```
GUIDES  # always on – walk me through it
```

```
RULES HOUSESTYLE SLIDES STRUCTURE GLOSSARY SUPPORT
```

```
RUNBOOKS  # on demand – run it, step by step
```

```
GETTING-STARTED TOUR LINT DOCTOR RED-TEAM STEELMAN
```

```
NEXT MIGRATE-CODE MIGRATE-STACK FROM-JUPYTER FROM-PAPER
```

```
EMBED-DOCS GROUND-CLAIMS UPDATE
```

Driven by a handful of commands

<code>task install</code>	set up (Python deps via uv)
<code>task run -- exp000</code>	run an experiment end-to-end
<code>task dev</code>	serve the site, live-reload on save
<code>task build</code>	website + a PDF per entry + a book
<code>task test</code>	run the test suite

Use any stack you want

- Ships set up for **Python**: `uv`, tools + runners as `.py`, `pytest`.
- But the contract is **files, not a language**: a tool just writes `numbers.json` + data.
- So switching is a conversation: “**migrate the stack to MATLAB**” — the agent follows a runbook.
- Works for **MATLAB · R · Julia · Octave**; the Typst publishing and the contract stay put.
- Keep Python only for plotting, or drop it entirely — your call.

Try it

demolab.eoinmurray.info

- Open a coding agent in a new, empty folder.
- Point it at the repo; approve as it sets things up.
- It hands you a live URL and your first experiment.

Hand it the loop, keep the science yours.